COMPLEMENTS OF NUMBERS

- 2's complement is obtained by adding 1 to 1's complement
- Example: 7 digits number (n=7)

2's complement of 1101100 = 0010011 + 1

= 0010100

2's complement of 0110111 = 1001000 + 1

= 1001001

i.e. The 2's complement is obtained by leaving the least significant 0's and the first 1 unchanged and then replacing 1's with 0's and 0's with 1's

"Complement of the complement restores the number"

$$(r^n - 1) - [(r^n - 1) - N] = N$$

$$r^n - [r^n - N] = N$$

SUBTRACTION WITH COMPLEMENTS

$$M - N = ?$$

1. Add the minuend "M" to the r's complement of subtrahend "N"
   - $M + (r^n - N) = M - N + r^n$
2. If $M \geq N$, the sum will produce an end carry and will be discarded
3. If $M < N$, then there will be no end carry in the sum and the sum will be
   - $r^n - (N - M)$, which is r's complement of $(N - M)$
   - So, take the r's complement of the sum and place negative sign in front

SUBTRACTION WITH COMPLEMENTS

Examples:

1. 72532 – 3250 = ? (M ≥ N)

M = 72532
10's complement of N = + 96750
169282
Discard the end carry M – N = 69282

2. 3250 – 72532 = ? (M < N)

M = 03250
10's complement of N = + 27468
30718
No end carry, so take
r's complement of the sum = 69282 and
place negative sign
M – N = – 69282

SUBTRACTION WITH COMPLEMENTS

Examples for binary numbers (same principle):

1010100 – 1000011= ? (M ≥ N)

M = 1 0 1 0 1 0 0

2's complement of N = + 0 1 1 1 1 0 1

1 0 0 1 0 0 0 1

Discard the end carry M – N = 1 0 0 0 1

1000011 – 1010100 = ? (M < N)

M = 1 0 0 0 0 1 1

2's complement of N = + 0 1 0 1 1 0 0

1 1 0 1 1 1 1

No end carry, so take

r's complement of the sum = 0 0 1 0 0 0 1 and place negative sign

M – N = – 1 0 0 0 1

SUBTRACTION WITH COMPLEMENTS

Subtraction of unsigned numbers can also be done using (r – 1)'s complement

(r – 1)'s complement is 1 less than r's complement,

So, the sum of minuend and the complement of subtrahend is 1 less than correct difference, when an end carry occurs

So, to get the correct difference remove the carry and add 1, i.e. ***end around carry***

1. 1010100 – 1000011= ?

An example of (M ≥ N)

M = 1 0 1 0 1 0 0

1's complement of N = + 0 1 1 1 1 0 0

1 0 0 1 0 0 0 0

If carry, end around carry

0 0 1 0 0 0 0

\+ 1

M – N = 0 0 1 0 0 0 1

SUBTRACTION WITH COMPLEMENTS

Subtraction of unsigned numbers can also be done using (r – 1)'s complement

When the sum of minuend and (r – 1)'s complement has no end carry then take (r – 1)'s complement of the sum and place minus sign

1. 1000011 – 1010100 = ?

An example of (M < N)

M = 1 0 0 0 0 1 1
1's complement of N = + 0 1 0 1 0 1 1
1 1 0 1 1 1 0

No end carry, so take
(r – 1) 's complement of the sum = 0 0 1 0 0 0 1 and place negative sign
M – N = – 1 0 0 0 1

SIGNED BINARY NUMBERS

- Positive integers (0, 1, 2, . . .) can be represented as **unsigned** numbers
- For negative numbers in ordinary arithmetic, we use minus sign
- However, Computer hardware must represent everything in bits
  - So, the left most bit is used to represent the sign
- It is important that representation is known in advance
  - Unsigned

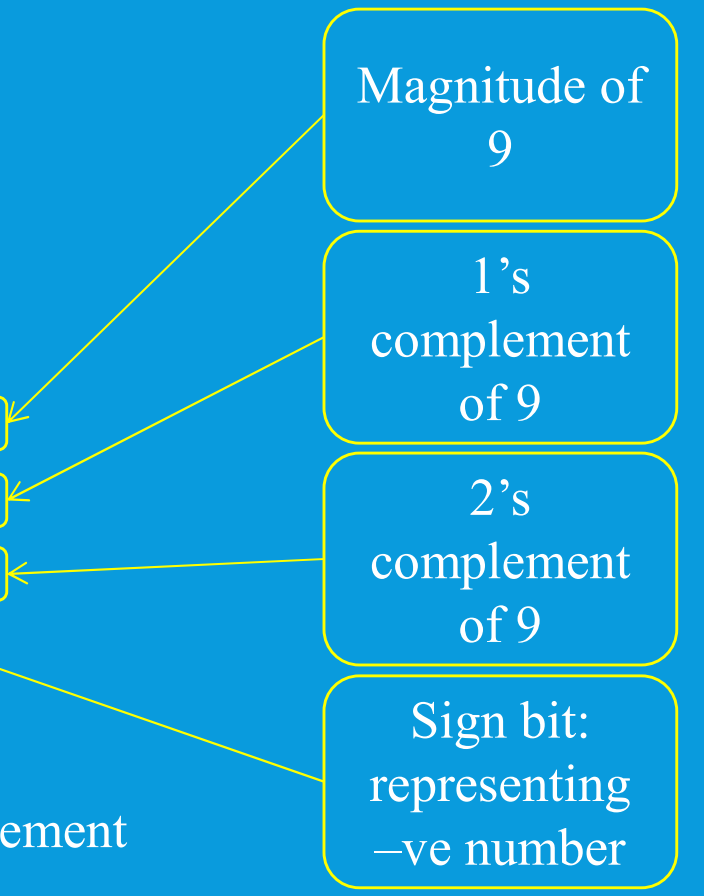
    $(01001)_2 = (9)_{10}$ $(11001)_2 = (25)_{10}$
  - Signed:

    $(01001)_2 = (9)_{10}$ $(11001)_2 = (-9)_{10}$

***Signed magnitude convention*: i.e. magnitude and the symbol (+ or –) or (0 or 1)**

SIGNED BINARY NUMBERS

- Signed magnitude convention is the one used in ordinary arithmetic
- However, in computers, arithmetic operations are implemented using ***signed complement system***
  - Convenient
  - Signed 1's complement (relatively difficult)
  - Signed 2's complement (Easier, more common)
- $(-9)_{10}$ represented in three systems (using 8 bits)
  - Signed magnitude representation: 10001001
  - Signed 1's complement representation: 11110110
  - Signed 2's complement representation: 11110111
- In all three systems
  - Left most bit represents (positive or negative sign)
  - The remaining part represents magnitude or complement



SIGNED BINARY NUMBERS

Positive numbers have same representation in all three systems

Left most bit (i.e. Most significant bit) represents sign in all three systems

Table 1.3
Signed Binary Numbers

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
−0	—	1111	1000
−1	1111	1110	1001
−2	1110	1101	1010
−3	1101	1100	1011
−4	1100	1011	1100
−5	1011	1010	1101
−6	1010	1001	1110
−7	1001	1000	1111
−8	1000	—	—

SIGNED BINARY NUMBERS

- Arithmetic addition
  - Ordinary arithmetic (sign magnitude system) requires comparison of signs and magnitudes
  - Signed complement system does not require comparison or subtraction

The addition of two signed binary numbers with negative numbers represented in signed- 2's-complement form is obtained from the addition of the two numbers, including their sign bits. A carry out of the sign-bit position is discarded.

SIGNED BINARY NUMBERS

- Arithmetic addition using signed-2's complement
  - Negative numbers must be in the form of signed-2's complement
  - The result is also obtained in the signed-2's complement form

```
+  6      0 0 0 0   0 1 1 0
+ 13      0 0 0 0   1 1 0 1
+19       0 0 0 1   0 0 1 1
```

```
+  6      0 0 0 0   0 1 1 0
– 13      1 1 1 1   0 0 1 1
–  7      1 1 1 1   1 0 0 1
```

```
–  6      1 1 1 1   1 0 1 0
+ 13      0 0 0 0   1 1 0 1
+  7    1 0 0 0 0   0 1 1 1
Carry discarded
Ans:      0 0 0 0   0 1 1 1
```

```
–  6      1 1 1 1   1 0 1 0
– 13      1 1 1 1   0 0 1 1
– 19    1 1 1 1 0   1 1 0 1
Carry discarded
Ans:      1 1 1 0   1 1 0 1
```

SIGNED BINARY NUMBERS

- Arithmetic addition
  - Buffer overflow

Overflow is a problem in computers because the number of bits that hold a number is finite, and a result that exceeds the finite value by 1 cannot be accommodated.

- If we start with two n-bit numbers and the sum occupies n + 1 bits, we say that an overflow has occurred
- Example: (4 bit signed number addition using signed-2's-complement)

+ 4	0 1 0 0
+ 5	0 1 0 1
+ 9	1 0 0 1

BUT, 1 0 0 1 is the representation of – 7 and not 9

- In order to obtain a correct answer, we must ensure that the result has a sufficient number of bits to accommodate the sum.

SIGNED BINARY NUMBERS

- Arithmetic subtraction using signed-2's complement

Take the 2's complement of the subtrahend (including the sign bit) and add it to the minuend (including the sign bit). A carry out of the sign-bit position is discarded.

$(\pm A) - (+B) = (\pm A) + (-B)$;

$(\pm A) - (-B) = (\pm A) + (+B)$;

  - i.e. we, change the subtraction operation to addition and
  - Subtrahend positive number is changed to negative and negative to positive by taking the 2's complement

- Example: (–6) – (–13) = (–6) + (+13)

– 6	1 1 1 1 1 0 1 0
+ 13	0 0 0 0 1 1 0 1
+ 7	1 0 0 0 0 0 1 1 1
Carry discarded	
Ans:	0 0 0 0 0 1 1 1

SIGNED BINARY NUMBERS

Why Signed-Complement System?

It is worth noting that binary numbers in the signed-complement system are added and subtracted by the same basic addition and subtraction rules as unsigned numbers. Therefore, **computers need only one common hardware circuit to handle both types of arithmetic.**

Why Signed-2's Complement is preferred over signed-1's complement?

BINARY CODES

- An n-bit binary **code** is an n-bit word which can represent up to 2^n different elements.
- Example: 3-bit code can represent up to 8 different elements"

All quantities in a PC needs to be expressed as a binary number: e.g. the digits 0, 1, …9; characters, control characters, etc. This can be done using a **"Code"**

Binary Codes

- **Example: A binary code for the seven colors of the rainbow**
- **Code 100 is not used**

Color	Binary Number
Red	000
Orange	001
Yellow	010
Green	011
Blue	101
Indigo	110
Violet	111

- **Given *n* binary digits (called bits), a binary code is a mapping from a set of represented elements to a subset of the 2^n binary numbers or elements.**

Binary Coded Decimal (BCD)

- **The BCD code is the 8,4,2,1 code.**
- **This code only encodes the first ten values from 0 to 9.**
- **Each decimal digit is coded separately by 4 bits**
- **Example:**
  - $(325)_{10} = (\underbrace{0011}_{3}\ \underbrace{0010}_{2}\ \underbrace{0101}_{5})_{BCD}$
- **Exercise: $(856)_{10} = (\quad)_{BCD}$**

Binary Coded Decimal (BCD)

- **BCD Addition:**

4	0 1 0 0
+ 5	0 1 0 1
9	1 0 0 1

4	0 1 0 0	
+ 8	1 0 0 0	
12	1 1 0 0	not a valid BCD number
	0 1 1 0	for correction add 6
	1 0 0 1 0	

$= (0001\ 0010)_{BCD}$

8	1 0 0 0	
+ 9	1 0 0 1	
17	1 0 0 0 1	end carry
	0 1 1 0	for correction add 6
	1 0 1 1 1	

$= (0001\ 0111)_{BCD}$

Other Decimal Codes

- **Weighted Codes**:
  - Each bit has associated weightage
  - e.g.
    - BCD (8421)
    - 2421 Codes:
      - Left most bit has weightage of 2
      - Right most bit has wieghtage of 1
- **Self complementing Codes**:
  - 9's complement of the decimal code is directly obtained by changing 1s to 0s and 0s to 1s
  - e.g.
    - 2421 Codes
    - Excess-3
      - Binary value + 3